

# 机器学习算法对比 平时作业报告

实验题目：基于鸢尾花二分类任务的多种分类算法由浅入深评测

学生姓名：侯景祺

学号：23071005

专业班级：230710

指导教师：梁音

完成时间：2026 年 5 月

## 目录

|                           |    |
|---------------------------|----|
| 摘要.....                   | 3  |
| 一、实验环境与算力配置.....          | 4  |
| 二、数据预处理与基础模型构建.....       | 4  |
| 2.1 数据集初步观察.....          | 4  |
| 2.2 基础模型构建.....           | 5  |
| 三、更深层次的分析与其他分类模型的引入.....  | 8  |
| 3.1 六类模型的基本思想.....        | 8  |
| 3.2 关于六类模型的实验.....        | 10 |
| 四、调用 GPU/XPU 的附加成本分析..... | 11 |
| 五、维度灾难与信息污染抗性分析.....      | 12 |
| 六、极小样本下的学习效率分析.....       | 13 |
| 七、模型可解释性与特征洞察.....        | 15 |
| 八、实验总结.....               | 17 |
| 参考文献.....                 | 18 |

## 摘要

本次实验一开始其实是围绕课程作业要求展开的，也就是从 Iris 鸢尾花数据集[2]中取前两类样本，完成一个二分类任务，并任选两种分类算法进行实现和比较。刚开始我选择的是两个风格差别很大的模型：一个是基于 PyTorch 实现的 MLP 神经网络[3]，另一个是表格数据任务中非常常用的 XGBoost 集成学习模型[4]。这样选择的原因也很直接：一个代表深度学习，一个代表传统机器学习中的强力树模型，我想看看它们在同一个简单数据集上到底有什么差别[1][2]。

但是实验真正跑起来之后，我发现事情没有想象中那么简单。MLP 和 XGBoost 在测试集上的准确率都达到了 100%，表面上看两个模型都非常优秀，但这也带来了一个问题：如果大家都是满分，那准确率就失去了区分能力。于是我没有停留在“跑通两个模型”这个层面，而是继续加入逻辑回归、线性 SVM、决策树和高斯朴素贝叶斯，组成了六类代表性模型，并进一步从决策边界、算力开销、维度噪声、小样本学习效率和特征重要性等角度进行分析。

通过这一系列实验，我逐渐意识到，模型比较不能只看最终准确率。对于简单二分类任务来说，准确率可能只是最表层的结果，真正能体现模型差异的，反而是训练效率、硬件调度成本、抗噪声能力、样本利用率和可解释性。最后的实验结果说明，没有绝对最强的算法，只有更适合当前数据规模、任务难度和工程条件的算法。

**关键词：**鸢尾花数据集；二分类；PyTorch；XGBoost；MLP；分类模型；模型可解释性；样本效率

# 一、实验环境与算力配置

一开始做这个实验的时候，我的目标其实比较简单，就是完成任选两种分类算法的作业要求。按照课程说明，本实验需要使用 Iris 数据集，并从三类鸢尾花中选取前两类，也就是 Setosa 和 Versicolor，构造一个二分类任务。这个任务本身是机器学习入门里非常经典的例子，因为前两类鸢尾花在特征空间中区分度比较高，适合用来观察不同分类算法的基本行为。

不过我不想只简单调用两个接口，然后输出一个准确率就结束。因为如果只是这样做，实验更像是在证明代码能跑，而不是在证明我真的理解了模型之间的差别。所以我一开始就选择了两个风格很不一样的算法：一个是用 PyTorch 搭建的 MLP 多层感知机，代表深度学习方法；另一个是 XGBoost，代表表格数据领域里非常强的集成树模型。我的想法是让这两个模型进行一次“硬碰硬”的对比，看神经网络和树模型在这个二分类任务上到底谁更有优势。

实验环境方面，我使用 Conda 来管理 Python 环境，并在本地环境中配置了 PyTorch、scikit-learn、XGBoost、NumPy、pandas 和 matplotlib 等常用库[3~5]。比较特别的是，我还成功调用了 Intel Arc 140V 显卡，也就是通过 PyTorch 的 XPU 后端让模型能够使用带 16GB 显存的显卡资源。虽然这个数据集本身并不大，但我还是想把硬件环境搭起来，因为这对后面分析 GPU/XPU 在小数据任务中的效率问题很有帮助。

表 1 实验环境与配置概况

| 类别   | 配置内容                                                 |
|------|------------------------------------------------------|
| 操作系统 | Windows 环境，本地终端执行实验代码                                |
| 环境管理 | Conda 环境管理，单独创建机器学习实验环境                              |
| 核心库  | PyTorch、scikit-learn、XGBoost、NumPy、pandas、matplotlib |
| 硬件加速 | Intel Arc 140V Graphics，PyTorch XPU 后端，显存约 16GB      |
| 数据集  | Iris 前两类样本，Setosa 与 Versicolor，100 samples           |

```
(ml_iris) PS E:\Iris> python
Python 3.11.15 | packaged by Anaconda, Inc. | (main, Mar 11 2026, 17:12:15) [MSC v.1942 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> import torch
>>> print("XPU is available:", torch.xpu.is_available())
XPU is available: True
>>> if torch.xpu.is_available():
...     print("Device Name:", torch.xpu.get_device_name(0))
...
Device Name: Intel(R) Arc(TM) 140V GPU (16GB)
>>>
```

图 1 实验环境与 XPU 调用验证截图

这张截图的意义不是单纯展示电脑配置，而是证明我确实完成了深度学习运行环境的配置。后面在比较 MLP 和 XGBoost 训练耗时的时候，我就可以进一步讨论一个很实际的问题：高算力设备在小数据任务上到底有没有必要。

# 二、数据预处理与基础模型构建

## 2.1 数据集初步观察

在正式训练模型之前，我先对数据本身做了一次简单观察。Iris 数据集虽然只有 4 个特征，但这 4 个特征并不是同等重要的。Setosa 和 Versicolor 在萼片长度、萼片宽度上有一定重叠，但在花瓣长度和花瓣宽度上差异非常明显。也就是说，在模型真正开始训练之前，数据本身已经给了一个很强的提示：花瓣相关特征很可能是完成分类的关键。

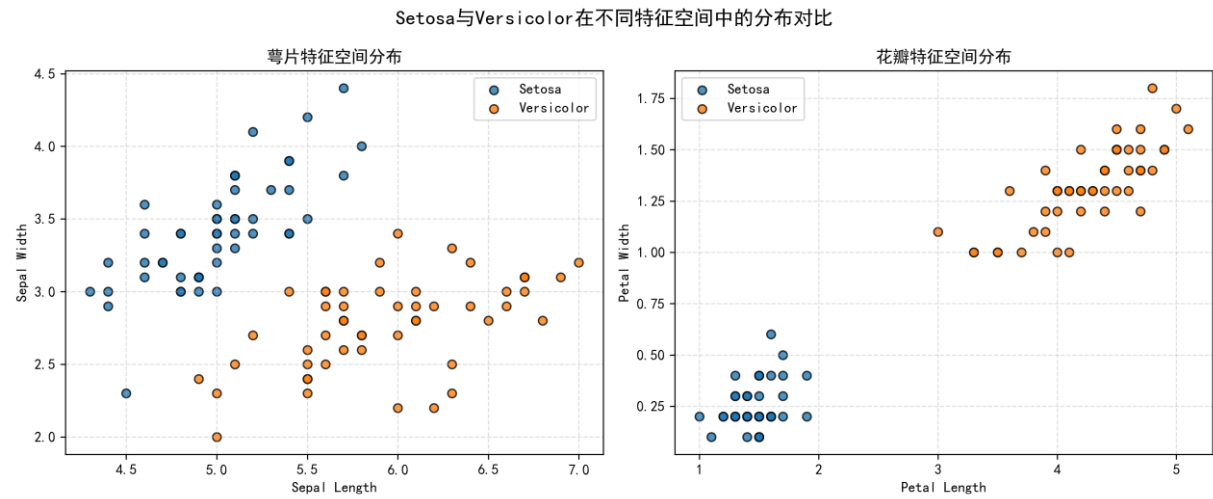


图 2-1 Setosa 与 Versicolor 在萼片和花瓣特征空间中的分布对比

这一点也解释了后面为什么很多模型都能达 100%准确率。并不是所有模型都天然非常强，而是当前二分类任务本身具有比较明显的可分性。如果不先观察数据分布，后面看到满分准确率时很容易误以为是模型能力特别突出；但结合数据分布来看，满分结果其实更像是“数据本身比较友好”。

表 2-1 二分类数据集类别数量统计

| 类别         | 样本数量 | 标签含义      |
|------------|------|-----------|
| Setosa     | 50   | 第一类鸢尾花    |
| Versicolor | 50   | 第二类鸢尾花    |
| 合计         | 100  | 二分类实验样本总数 |

表 2-2 四个特征的均值和标准差统计

| 类别         | Sepal Length 均值±标准差 | Sepal Width 均值±标准差 | Petal Length 均值±标准差 | Petal Width 均值±标准差 |
|------------|---------------------|--------------------|---------------------|--------------------|
| Setosa     | 5.006±0.352         | 3.428±0.379        | 1.462±0.174         | 0.246±0.105        |
| Versicolor | 5.936±0.516         | 2.770±0.314        | 4.260±0.470         | 1.326±0.198        |

2.2 基础模型构建

完成环境配置之后，我先按照作业要求处理 Iris 数据集。原始 Iris 数据集一共有三类鸢尾花，每类 50 条样本，每条样本包含 4 个特征，分别是萼片长度、萼片宽度、花瓣长度和花瓣宽度。本实验只取前两类 Setosa 和 Versicolor，因此最终得到的是一个 100 条样本的二分类数据集。

在数据预处理阶段，我主要做了三件事。第一步是读取原始数据，把字符串标签转换成数字标签；第二步是筛选前两类样本，去掉第三类 *Virginica*；第三步是进行特征标准化。标准化这一步很重要，因为 MLP、逻辑回归、SVM 这类模型都比较依赖特征尺度，如果某个特征数值范围明显更大，就可能在训练中占据过多影响。

#### 核心伪代码 1 数据读取、类别筛选、训练测试划分与标准化

```
# 读取 Iris 数据集
X, y = load_iris_data()

# 只保留前两类: Setosa 和 Versicolor
X_binary = X[y != 2]
y_binary = y[y != 2]

# 划分训练集和测试集
X_train, X_test, y_train, y_test = train_test_split(
    X_binary, y_binary,
    test_size=0.2,
    random_state=42,
    stratify=y_binary
)

# 对特征进行标准化
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

基础模型部分，我先实现了 PyTorch MLP。MLP 的结构并不复杂，本质上就是输入层接收 4 个特征，中间通过若干个隐藏层进行非线性变换，最后输出二分类结果。它的优势是表达能力强，可以学习非线性关系；但缺点也很明显，就是需要反复迭代训练，对数据规模和训练参数比较敏感。

#### 核心代码 1 PyTorch MLP 网络结构与训练循环

```
class MLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(4, 16),
            nn.ReLU(),
            nn.Linear(16, 8),
            nn.ReLU(),
            nn.Linear(8, 2)
        )

    def forward(self, x):
        return self.net(x)

for epoch in range(num_epochs):
    optimizer.zero_grad()
    outputs = model(X_train_tensor)
    loss = criterion(outputs, y_train_tensor)
    loss.backward()
    optimizer.step()
```

```

● (ml_iris) PS E:\Iris> python .\mlp_iris.py
torch MLP Iris实验
当前计算设备: xpu
训练耗时: 672.77 ms
测试集准确率: 100.00%

```

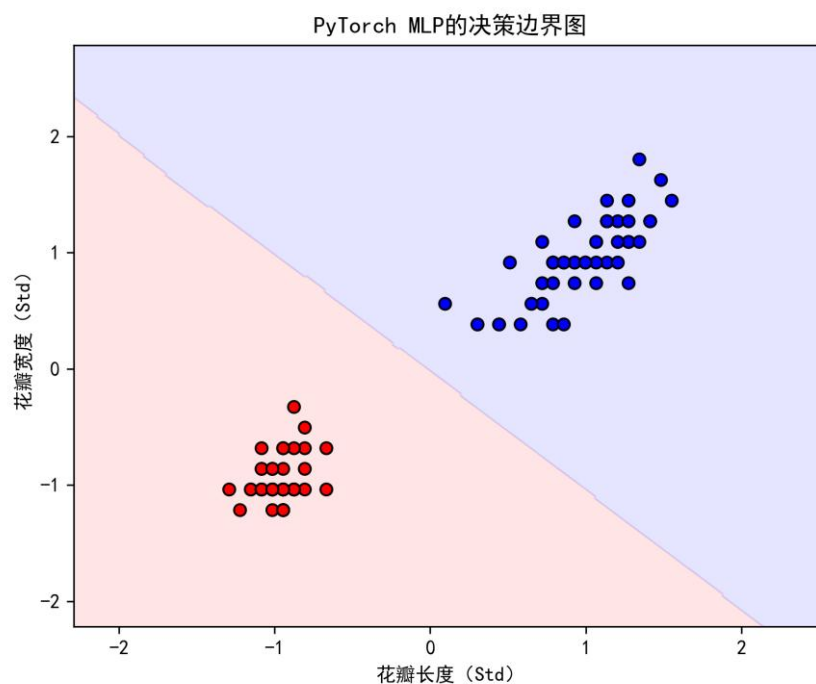
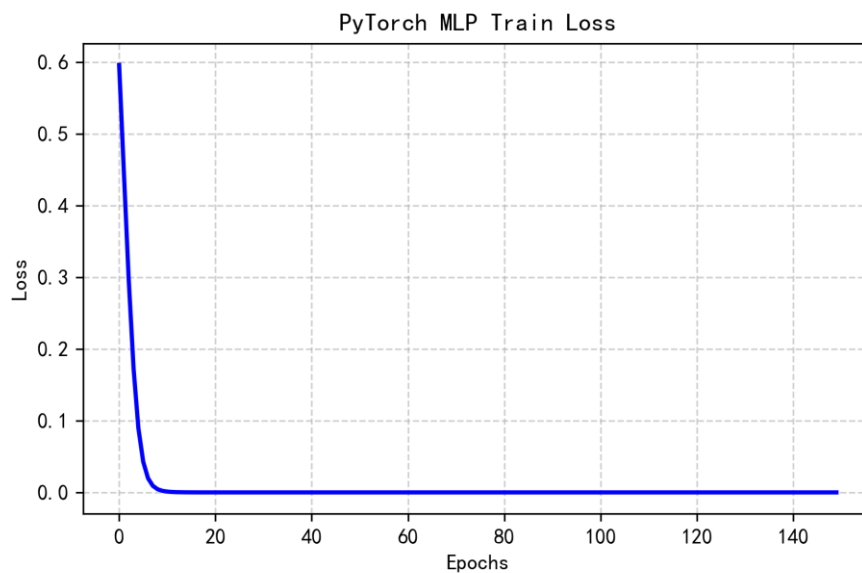


图 2 PyTorch MLP 效果截图

与 MLP 相对应，我又调用了 XGBoost 模型。XGBoost 不像神经网络那样通过反向传播更新大量权重，而是通过不断构建树模型来修正前一轮的错误。对于表格数据来说，XGBoost 往往非常强，因为它可以自动发现特征分裂点，也不太依赖复杂的数据标准化。

#### 核心代码 2 XGBoost 模型调用与准确率计算

```

model = XGBClassifier(
    n_estimators=100,
    max_depth=3,
    learning_rate=0.1,

```

```

eval_metric="logloss",
random_state=42
)

model.fit(X_train, y_train)
y_pred = model.predict(X_test)
acc = accuracy_score(y_test, y_pred)
print("XGBoost Accuracy:", acc)

```

```

(m1_iris) PS E:\Iris> python .\xgb_iris.py
XGBoost集成学习Iris实验
C:\Users\LC\miniconda3\envs\m1_iris\Lib\site
st\xgboost\src\learner.cc:782:
Parameters: { "use_label_encoder" } are not

    bst.update(dtrain, iteration=i, fobj=obj)
训练耗时: 11.45 ms
测试集准确率: 100.00%

```

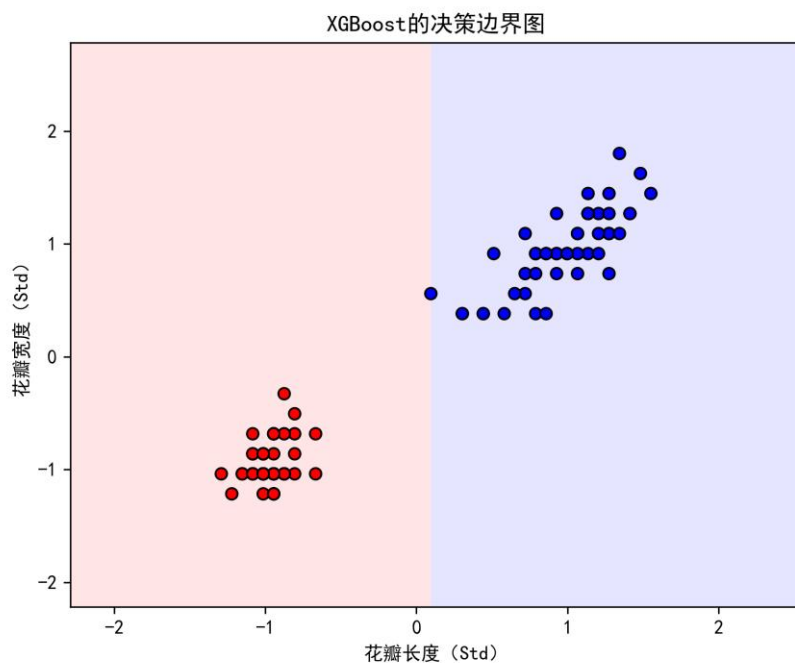


图 3 XGBoost 核心代码与运行结果截图

这一步完成之后，基础任务其实已经跑通了。按照作业要求来说，两个分类算法、核心代码、环境截图和结果截图都已经具备。但是后面的结果让我意识到，这个实验如果只做到这里，分析还远远不够。

### 三、更深层次的分析与其他分类模型的引入

#### 3.1 六类模型的基本思想

模型训练完成后，我最开始期待的是 MLP 和 XGBoost 会出现比较明显的差距。比如我以为神经网络可能因为结构复杂而更强，或者 XGBoost 因为适合表格数据而更稳。但真正跑完之后，我发现两个模型的测试集准确率都是 100%。

这个结果一开始让我有一点诧异，但很快又有点无聊。模型确实都完成了分类任务；但如果两个模型都是满分，那准确率就没有办法继续说明谁更好。后来我回到数据本身去看，才发现原因其实很直接：Iris 前两类 Setosa 和 Versicolor 在特征空间上本来就非常容易分开，尤其是花瓣长度和花瓣宽度这两个特征，已经足够把两类样本区分开。

也就是说，这里的 100%准确率并不一定说明模型有多聪明，更可能说明这个任务本身太简单。准确率表面上很好看，但它掩盖了模型之间真正的差异。我把这种现象理解成一种虚假的繁荣：分数很高，但分析价值反而下降了。

为了打破这个局面，我决定把实验继续扩展，不再只比较 MLP 和 XGBoost，而是额外加入逻辑回归、线性 SVM、决策树和高斯朴素贝叶斯。这样一来，实验就从“两种算法对比”扩展成了“六大经典模型比较”。

表 2 六类模型及其比较意义

| 模型                   | 所属方法族       | 引入目的                 |
|----------------------|-------------|----------------------|
| Logistic Regression  | 线性模型        | 观察最基础的线性分类边界         |
| Linear SVM           | 最大间隔线性模型    | 观察线性分隔与间隔最大化思想       |
| Decision Tree        | 树模型         | 观察基于阈值的阶梯状划分         |
| Gaussian Naive Bayes | Naive Bayes | 观察小样本下的概率建模优势        |
| PyTorch MLP          | 神经网络        | 代表深度学习方法，观察非线性拟合能力   |
| XGBoost              | 集成学习        | 代表强力表格模型，观察抗噪和特征筛选能力 |

为了让后面的比较不是简单“多跑几个模型”，我又把每个模型背后的思路重新梳理了一遍。

逻辑回归虽然名字里有“回归”，但在这里它本质上是一个线性分类器。它先把四个输入特征做线性组合，再通过 sigmoid 函数把结果压缩到 0 到 1 之间。它的特点是结构简单、训练速度快，而且系数本身比较容易解释。[6]

线性 SVM 的关注点不是概率，而是间隔。它希望找到一条分界线，不只是把两类样本分开，还要让样本离分界线尽量远。所以 SVM 更像是在追求一个“更稳”的边界。[7]

决策树的思路则完全不同。它不是一次性画出一条整体边界，而是不断问问题：某个特征是否小于某个阈值？如果是就走左边，不是就走右边。这样一层层判断下去，最后得到分类结果。[8]

高斯朴素贝叶斯属于概率模型。它会假设每一类样本的特征大致服从高斯分布，然后根据概率来判断一个新样本属于哪一类。这个假设并不总是完全成立，但它非常轻量，在小样本场景下经常能有不错表现。

MLP 神经网络的优势是可以通过多层非线性变换学习复杂关系，但它也需要更多训练过程和参数调整。对于很小的数据集来说，神经网络未必一定比传统模型更划算。[9]

XGBoost 则是集成学习中的代表模型。它通过一棵棵树不断修正前面的错误，既能处理非线性关系，也能自动筛选有用特征，因此在表格数据任务中经常表现很强。[4]

## 3.2 关于六类模型的实验

### 核心伪代码 2 六大模型统一训练与测试流程

```
models = {  
    "Logistic Regression": LogisticRegression(),  
    "Linear SVM": SVC(kernel="linear"),  
    "Decision Tree": DecisionTreeClassifier(),  
    "Gaussian Naive Bayes": GaussianNB(),  
    "PyTorch MLP": TorchMLP(),  
    "XGBoost": XGBClassifier()  
}  
  
for name, model in models.items():  
    model.fit(X_train, y_train)  
    y_pred = model.predict(X_test)  
    acc = accuracy_score(y_test, y_pred)  
    print(name, acc)
```

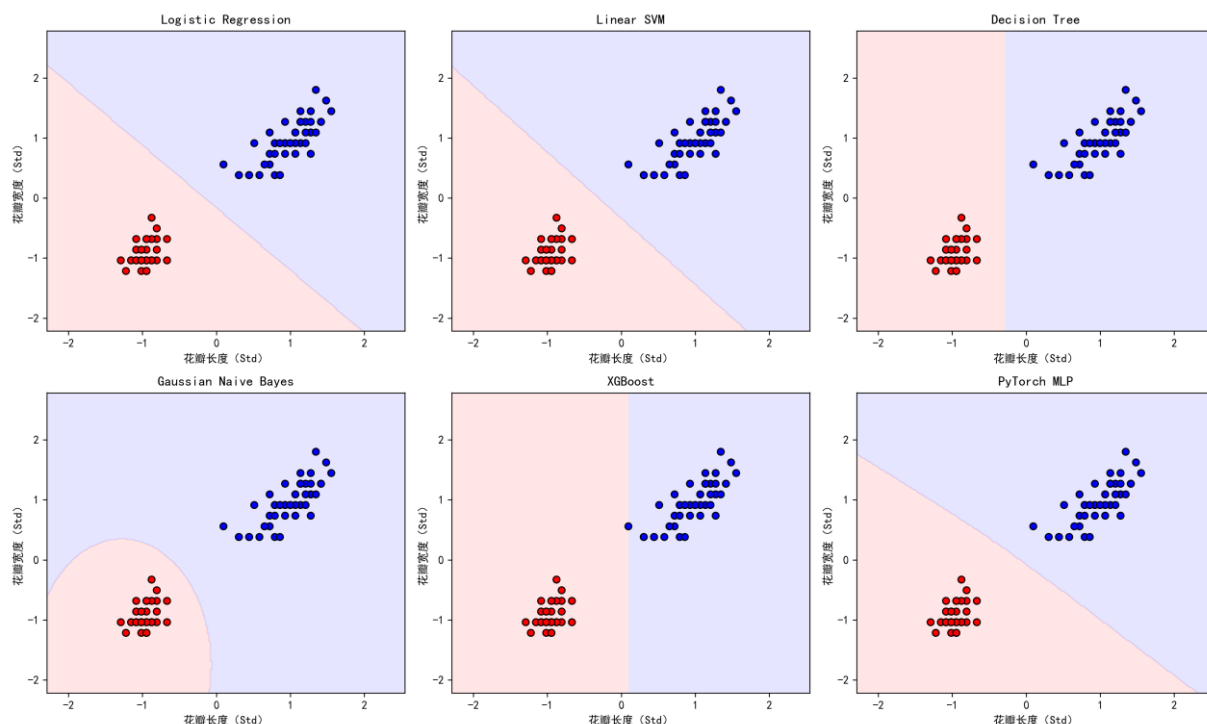


图 4 六大模型决策边界

从决策边界图里可以看出，虽然这些模型在这个二分类任务上的准确率都很高，但它们工作的方式其实并不一样。逻辑回归和线性 SVM 的边界最直观，基本就是在特征空间里画出一条斜直线，把两类样本分到两边，体现的是线性分类模型“一刀切”的思路。决策树和 XGBoost 的边界则更像是按照某个特征阈值直接切开，尤其在这张图里，它们主要依赖花瓣长度这一类关键特征，所以边界呈现出比较明显的竖直分割，而不是平滑斜线。高斯朴素贝叶斯的边界比较

特殊，它不是单纯画直线，而是根据每一类样本的概率分布来判断，因此会出现带弧度的曲线区域。PyTorch MLP 虽然属于神经网络模型，但由于当前两类鸢尾花本身非常容易分开，所以它学到的边界也接近一条较平滑的斜线，并没有表现出特别复杂的非线性形状。也就是说，图中最值得关注的并不是谁分得更准，而是同样能分对的情况下，不同算法背后的分类逻辑其实完全不同。

这一步让我真正意识到，模型比较不能只看最终分数。就算准确率一样，不同模型背后的判断方式、边界形状和泛化逻辑也可能完全不同。

## 四、调用 GPU/XPU 的附加成本分析

在基础实验中，我还发现了另一个非常反常的现象：使用 Intel Arc 140V 显卡运行的 PyTorch MLP，居然比 CPU 上运行的 XGBoost 慢了很多，甚至慢了大约 74 倍。

刚看到这个结果时，我其实有点疑惑。因为从直觉上讲，GPU 或 XPU 应该是更强的算力设备，尤其是还有 16GB 显存，看起来应该比 CPU 更适合跑模型。但这个实验的数据集只有 100 条样本，训练集也就 80 条左右。对于这么小的数据来说，显卡真正用于计算的时间非常少，更多时间反而消耗在数据搬运、设备调度、张量转换和运行框架开销上。

换句话说，这就像用一台大型挖掘机去夹一粒米。机器本身很强，但启动、调度和移动的成本太高，反而不如直接用手拿更快。这里的显卡也是一样，它不是不强，而是在这个任务上完全没有发挥空间。

为了验证这一点，我设计了一个“算力拐点测试”。具体做法是不断复制训练数据，把数据量从几十条逐渐扩大到几千、几万甚至十万级别，然后观察 MLP 和 XGBoost 的耗时变化。

### 核心伪代码 3 算力拐点测试

```
repeat_factors = [1, 10, 100, 1000, 5000]

for r in repeat_factors:
    X_big = np.tile(X_train, (r, 1))
    y_big = np.tile(y_train, r)

    mlp_time = train_and_time_pytorch_mlp(X_big, y_big, device="xpu")
    xgb_time = train_and_time_xgboost(X_big, y_big)

    print(r, len(X_big), mlp_time, xgb_time)
```

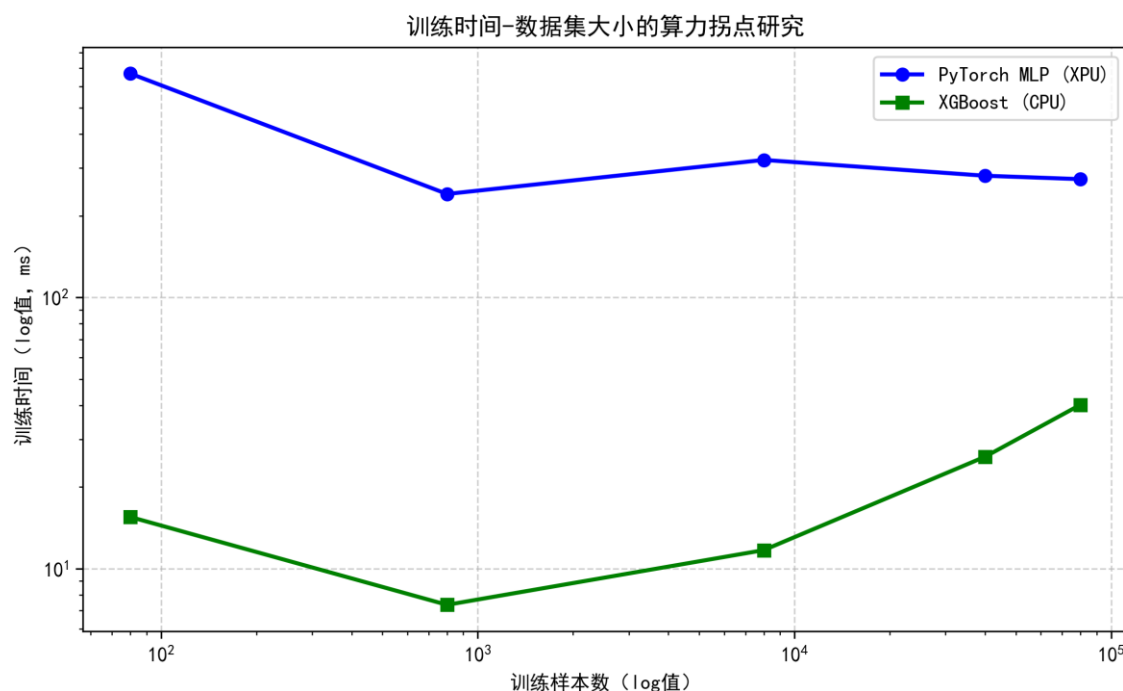


图 5 算力拐点测试图

从结果看，在原始的 80 条训练样本下，XPU 几乎没有发挥优势，反而因为调度开销变得很慢。即使我把数据人工复制到更大的规模，CPU 上的 XGBoost 依然表现得非常高效。这个结果说明，小数据任务并不是越上显卡越好。对于 Iris 这种小型表格数据来说，CPU 运行传统机器学习模型往往更加直接、更加轻量。

这一节给我的最大启发是：算力不是免费的。GPU/XPU 的使用本身也有成本，只有当数据量和计算量足够大时，这种成本才有可能被摊薄。否则，所谓的高配置反而可能变成沉没成本。

## 五、维度灾难与信息污染抗性分析

既然干净数据集上大家都能拿 100 分，我接下来就想测试一个更接近现实的问题：如果数据没有这么干净会怎样？

现实中的表格数据往往不是只有几个干净特征，而是可能夹杂大量无用字段、冗余字段甚至随机噪声。比如有些业务数据里，真正有用的字段可能只有几个，但表格里却有几百个变量。如果模型不能识别哪些特征有用，就很容易被这些垃圾信息干扰。[6][10]

所以我设计了一个维度灾难测试：在原本 4 个鸢尾花特征后面，强行加入大量随机噪声特征。比如加入 0 个、10 个、50 个、200 个完全随机的无用特征，再观察不同模型准确率会不会下降。

### 核心伪代码 4 随机噪声特征注入

```
noise_dims = [0, 10, 50, 100, 200]

for dim in noise_dims:
    noise_train = np.random.randn(X_train.shape[0], dim)
    noise_test = np.random.randn(X_test.shape[0], dim)
```

```

X_train_noise = np.hstack([X_train, noise_train])
X_test_noise = np.hstack([X_test, noise_test])

for name, model in models.items():
    model.fit(X_train_noise, y_train)
    y_pred = model.predict(X_test_noise)
    acc = accuracy_score(y_test, y_pred)
    print(name, dim, acc)

```

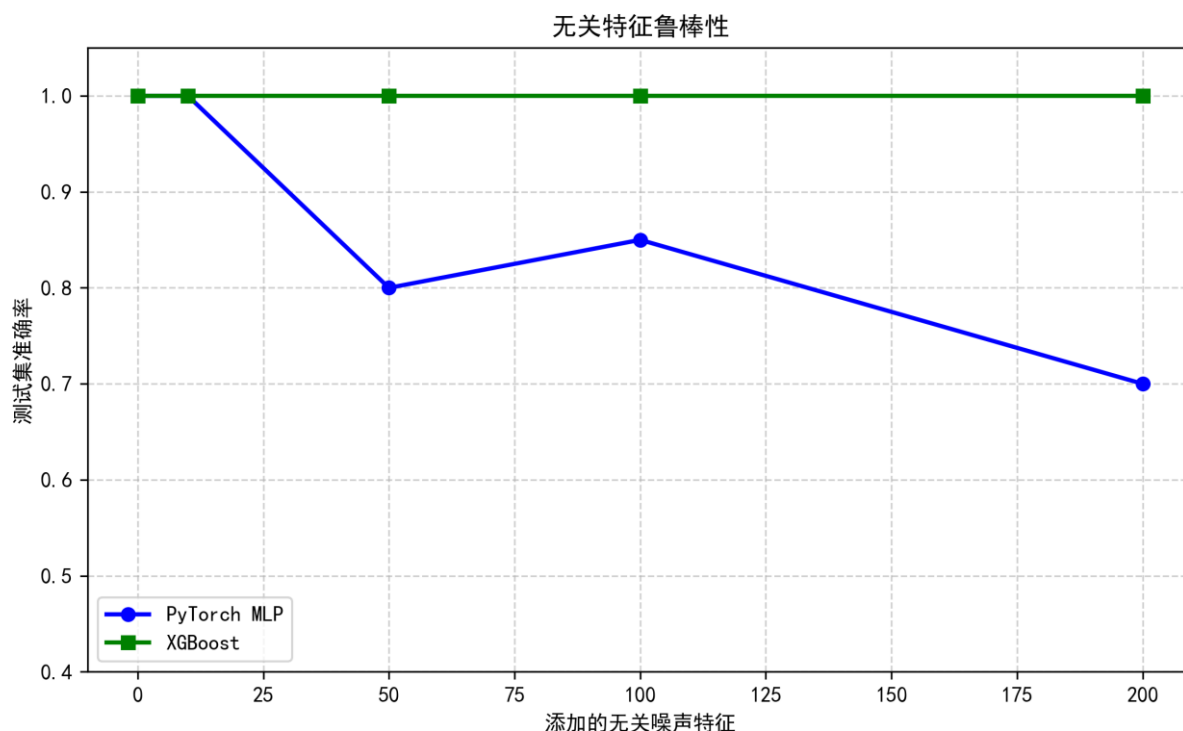


图 6 维度灾难测试图

这组实验的结果非常有意思。随着垃圾特征数量增加，一些模型开始明显不稳定，尤其是 MLP 和逻辑回归这类会同时处理所有输入维度的模型，很容易受到无用特征干扰。它们会试图在高维空间里寻找规律，但随机噪声本身没有规律，于是模型就可能出现过拟合或者边界震荡。

相比之下，XGBoost 表现得非常稳。即使加入大量垃圾特征，它仍然能保持很高的准确率。原因在于树模型每次分裂时都会选择能带来最大信息增益的特征。如果某些特征本身是随机噪声，它们很难稳定提供有效分裂，因此自然会被模型忽略。

这一点让我对树模型有了更深的理解。XGBoost 强不只是因为它能拟合复杂关系，还因为它在表格数据中具有很强的特征筛选能力。对于含有大量无关变量的数据，树模型天生就有一定对垃圾特征的免疫力。

## 六、极小样本下的学习效率分析

前面测试的是特征太多会怎样，接下来我又想反过来测试数据太少会怎样。

深度学习经常被说成是需要很多很多数据才能 train，也就是说它通常需要比较多的数据才能稳定训练。但 Iris 数据集本来就很小，如果我再故意减少训练样本，不同模型会不会表现出完全不同的生存能力？于是我设计了样本效率实验。

具体做法是只给模型一部分训练数据。其中 5% 的训练比例非常极端，可能只有 4 条左右训练样本。在这种 Few-Shot 状态下，模型基本处于没法吃饱的状态，我想看谁还能尽量保持稳定。

#### 核心伪代码 5 小样本训练比例测试

```
train_ratios = [0.05, 0.1, 0.2, 0.4, 0.6, 0.8, 1.0]

for ratio in train_ratios:
    X_small, _, y_small, _ = train_test_split(
        X_train, y_train,
        train_size=ratio,
        stratify=y_train,
        random_state=42
    )

    for name, model in models.items():
        model.fit(X_small, y_small)
        y_pred = model.predict(X_test)
        acc = accuracy_score(y_test, y_pred)
        print(name, ratio, acc)
```

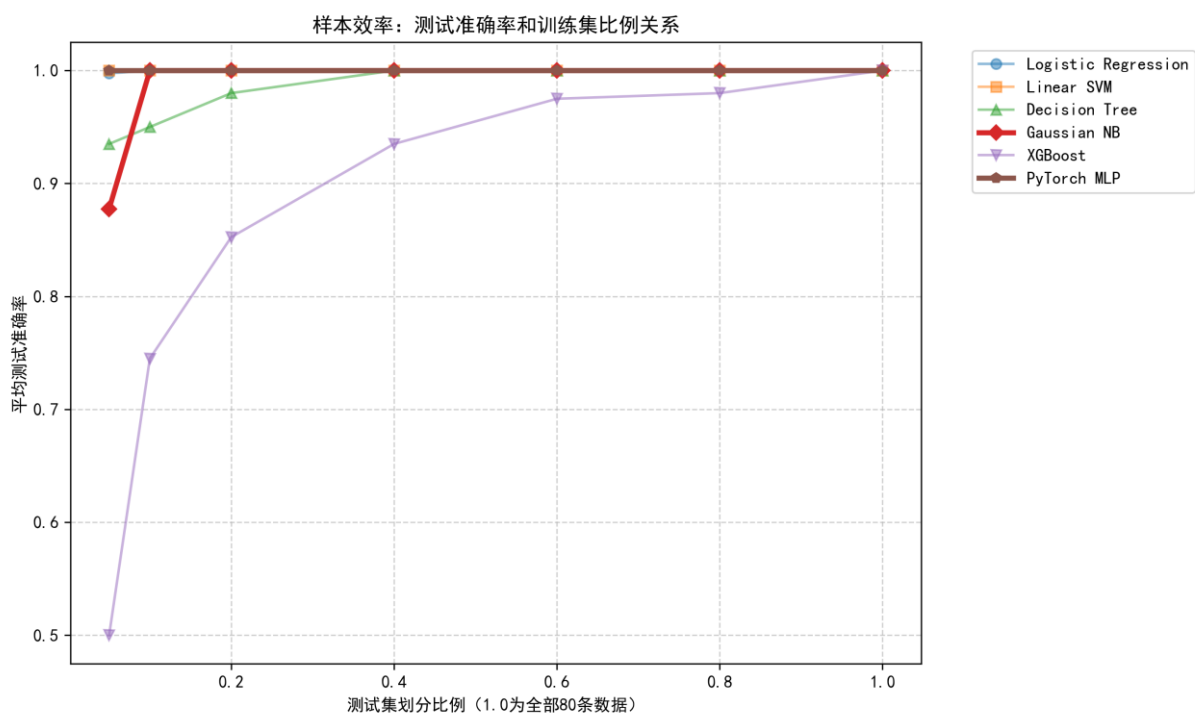


图 7 样本效率折线图

结果显示，在这组样本效率测试中，不同模型对训练数据比例的敏感程度并不一样。逻辑回归、线性 SVM 和 PyTorch MLP 的曲线几乎一直贴近 1.0，说明在这个二分类任务里，只要给到少量样本，它们就已经能够学到比较稳定的分界线。这也再次说明，Setosa 和 Versicolor 本身区分度很高，尤其在花瓣长度、花瓣宽度这类特征上，两类样本已经比较容易分开，所以线性模型和简单神经网络并不需要大量数据就能完成分类。

相比之下，XGBoost 在极小样本比例下表现反而不稳定。尤其在训练比例只有 5% 时，准确率明显偏低，随着训练样本比例增加，准确率才逐步上升，最后接近 100%。这个现象说明，集成树模型虽然在表格数据中通常很强，但它也需要足够的样本来支持树的分裂和 boosting 迭代。如果样本太少，每个特征阈值下可供判断的数据点非常有限，模型就很容易学不到稳定的分裂规则。换句话说，XGBoost 不是能力不强，而是在极小样本条件下还没有足够的信息让它发挥优势。

高斯朴素贝叶斯和决策树的表现介于两者之间。高斯朴素贝叶斯在 5% 样本时没有直接达到满分，但当样本比例稍微增加后很快稳定到 100%，说明它的概率假设在小样本下仍然比较有效。决策树也随着样本增加逐渐提升，但在低比例训练数据下略有波动，原因是单棵树对训练样本分布比较敏感，样本太少时容易受到个别点影响。

这组实验让我意识到，样本效率不能简单理解成：传统模型一定更强 或者 深度学习一定更吃数据。在当前这个 Iris 前两类二分类任务中，由于数据本身非常容易分开，简单线性模型和 MLP 反而很快就能学到有效边界；而 XGBoost 这种依赖树分裂和集成迭代的模型，在极小样本时反而需要更多数据来稳定发挥。也就是说，模型是否省数据，不只取决于算法家族，还和数据本身的可分性、样本规模以及模型的学习机制有关。

## 七、模型可解释性与特征洞察

经过前面的实验，我已经看到了不同模型在准确率、效率、噪声和小样本条件下的差异。但最后我还有一个问题：这些模型到底是根据什么做出判断的？

如果模型只是输出一个 0 或 1，我只能知道它分对了还是分错了，却不知道它为什么这么分。为了打开这个黑盒，我又做了特征重要性和特征权重分析。

对于线性模型，比如逻辑回归和线性 SVM，可以直接查看每个特征的系数。系数为正，说明这个特征越大，越倾向于预测为某一类；系数为负，则说明这个特征越大，越倾向于另一类。对于 XGBoost 和决策树，可以查看特征重要性，看看模型在分裂时最依赖哪些特征。[4][8]

### 核心伪代码 6 特征权重与重要性提取

```
# 线性模型系数
linear_weights = logistic_model.coef_

# XGBoost 特征重要性
xgb_importance = xgb_model.feature_importances_

# 决策树特征重要性
tree_importance = tree_model.feature_importances_
```

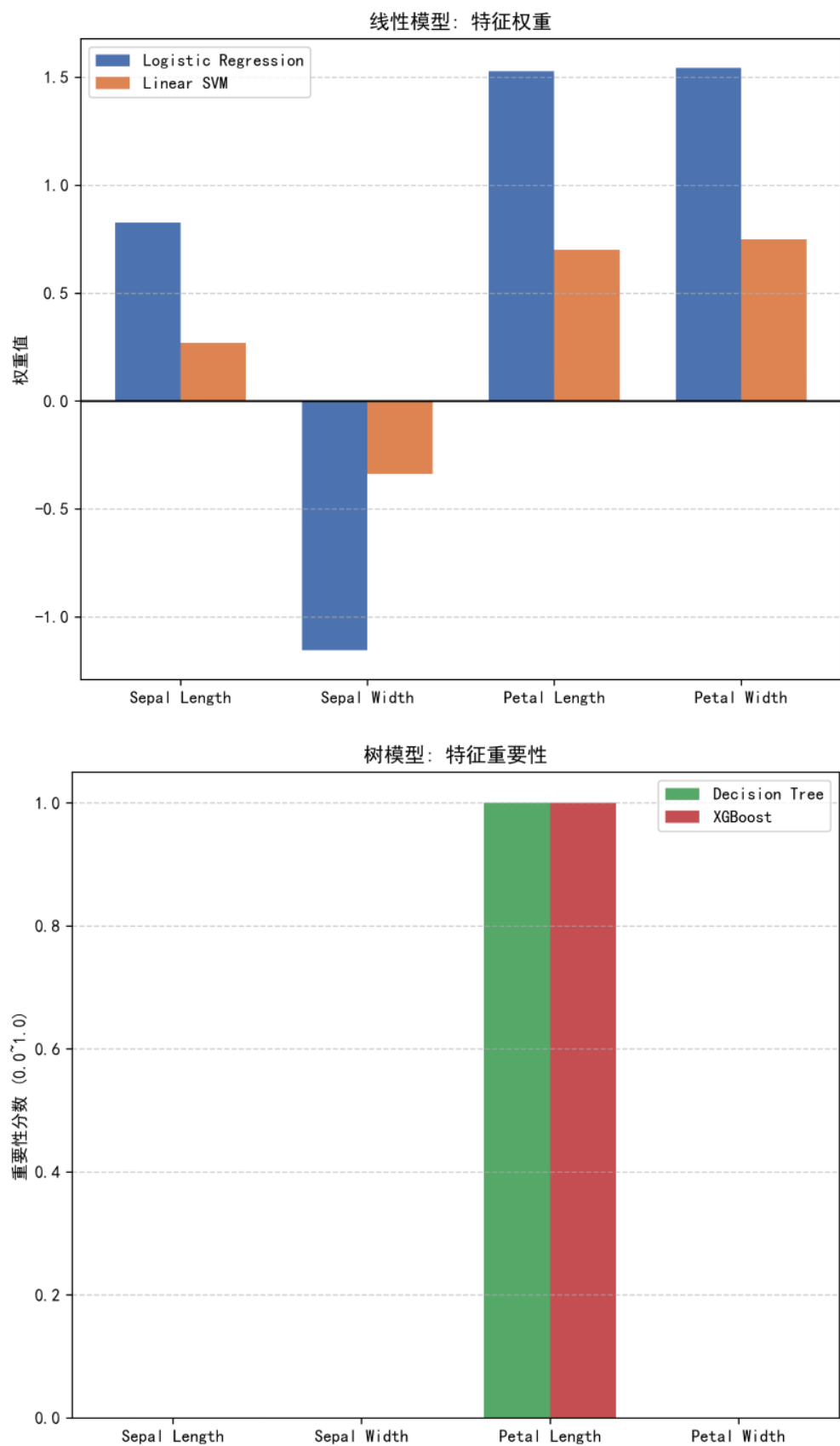


图 8 特征权重与重要性对比图

结果比较直观，但也不能简单理解成：所有模型都同时重视花瓣长度和花瓣宽度。从上半部分线性模型的特征权重可以看出，逻辑回归和线性 SVM 会综合利用四个特征来判断类别：

Sepal Length、Petal Length 和 Petal Width 的权重为正，说明这些特征数值越大，模型越倾向于判为 Versicolor；而 Sepal Width 的权重为负，说明萼片宽度较大时，模型更倾向于判为 Setosa。也就是说，线性模型不是只盯着某一个特征，而是在整体权衡四个特征对分类方向的影响。

下半部分树模型的特征重要性则更加极端。在这次实验结果中，决策树和 XGBoost 都把特征重要性几乎全部集中在 Petal Length 上，其他三个特征的重要性接近 0。这说明在当前的 Setosa 和 Versicolor 二分类任务里，花瓣长度已经足够形成非常有效的分裂规则。对于树模型来说，只要 Petal Length 这一个特征就能把两类样本基本切开，继续使用 Petal Width 或萼片特征带来的额外收益就很小，所以它们没有被赋予明显的重要性。

这个结果也和 Iris 数据本身的规律是一致的。Setosa 和 Versicolor 的主要差异确实集中在花瓣相关特征上，而在本次划分和训练结果中，Petal Length 表现得最关键。需要注意的是，这并不代表 Petal Width 在植物学或所有模型中都没有意义，而是说明在当前这个二分类实验里，树模型认为只看花瓣长度已经足够完成分类。

这一节让我觉得可解释性非常重要。准确率只能告诉我模型有没有分对，但特征权重和特征重要性可以进一步告诉我模型为什么能分对。尤其是树模型把判断依据集中到 Petal Length 上之后，模型的分类逻辑就变得非常清楚：它不是神秘地算出了结果，而是抓住了两类鸢尾花之间最明显的花瓣长度差异。

## 八、实验总结

回顾整个实验，我最开始只是想完成一个二分类作业，选择两种模型进行比较。但随着实验一步步深入，我发现这个任务比表面上更有分析价值。

一开始，MLP 和 XGBoost 都拿到了 100% 准确率，这看起来像是一个非常完美的结果。但后来我意识到，正是因为准确率太完美，才暴露出单一指标的局限性[6][10]。当前两类鸢尾花本身就很容易分开时，很多模型都能拿到满分，这时候准确率已经不能说明模型之间真正的区别。

于是我继续补充了更多模型和更多测试维度。通过六大模型决策边界可视化，我看到了不同算法完全不同的分类思路；通过算力拐点测试，我理解了小数据任务中 GPU/XPU 调度成本可能反而成为负担；通过维度灾难测试，我看到了 XGBoost 在无用特征干扰下的鲁棒性；通过样本效率测试，我发现高斯朴素贝叶斯等传统模型在小样本场景下依然有不可替代的优势；通过可解释性分析，我进一步看清了模型主要依赖花瓣长度和花瓣宽度进行分类。

所以这次实验给我的最大收获是：模型评估不能只看准确率。准确率只是模型表现的一个表象，真正完整的评价应该同时考虑效率、鲁棒性、样本利用率、硬件成本和可解释性。深度学习和 GPU 并不是万能答案，传统机器学习模型也并没有过时。不同算法各有适用场景，关键不是盲目选择最复杂的模型，而是根据数据特点和任务需求选择最合适的模型。

## 参考文献

- [1] Fisher, R. A. The use of multiple measurements in taxonomic problems. *Annals of Eugenics*, 1936, 7(2):179-188.
- [2] Dua, D., Graff, C. UCI Machine Learning Repository: Iris Data Set. University of California, Irvine, School of Information and Computer Sciences. <http://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data>
- [3] Paszke, A., Gross, S., Massa, F., et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library. *Advances in Neural Information Processing Systems*, 2019.
- [4] Chen, T., Guestrin, C. XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016:785-794.
- [5] Pedregosa, F., Varoquaux, G., Gramfort, A., et al. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 2011, 12:2825-2830.
- [6] Bishop, C. M. *Pattern Recognition and Machine Learning*. Springer, 2006.
- [7] Cortes, C., Vapnik, V. Support-Vector Networks. *Machine Learning*, 1995, 20(3):273-297.
- [8] Breiman, L., Friedman, J. H., Olshen, R. A., Stone, C. J. *Classification and Regression Trees*. Wadsworth, 1984.
- [9] Goodfellow, I., Bengio, Y., Courville, A. *Deep Learning*. MIT Press, 2016.
- [10] Hastie, T., Tibshirani, R., Friedman, J. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2nd ed. Springer, 2009.